Numerical examples from Excel — analytical condition checks

Representative exact-class parameter sets extracted from the manuscript v3 Excel datasets. The notebook computes derived Michaelis constants, detached-branch inequalities, and the Class 6–8 crossing-slope check. These examples illustrate the derivations; they do not re-label the full dataset.

---

1. Source and selection of examples

Examples are taken from the manuscript v3 exact-class Excel datasets, sheet ML_Ready_Data. One representative row per exact class is selected near the class median of k2; Classes 7 and 8 come from iteration 4 so their asymmetric first-site parameters are represented.

In[211]:=

```
sourceNote =
  "Representative rows from ML_Ready_Dataset_v3_iter1.xlsx through iter4.xlsx;
    one row per Exact_Class_v3, sheet ML_Ready_Data.
    Selected row numbers are preserved in the table.";
sourceNote
```

Out[212]=

```
Representative rows from ML_Ready_Dataset_v3_iter1.xlsx
  through iter4.xlsx; one row per Exact_Class_v3, sheet
  ML_Ready_Data. Selected row numbers are preserved in the table.
```

Added for the figures: set-up for the plot cells of this notebook. Evaluate it before any plot cell below. All names begin with "fig", so the derivations above and below are not affected.

In[213]:=

```
(* Shared set-up for the plot cells: evaluate this cell once, before any plot cell.
   All names start with "fig" so that they do not clash with the symbols of the
   derivation notebooks (k1, A, K, P, u, x, alpha, ... are left untouched).
   Every figure is shown and also exported as PDF to the sub-folder "plots"
   next to this notebook (or of the working directory if the notebook is unsaved). *)
figDir = Quiet[Check[FileNameJoin[{NotebookDirectory[], "plots"}], $Failed]];
If[! StringQ[figDir], figDir = FileNameJoin[{Directory[], "plots"}]];
If[! DirectoryQ[figDir], CreateDirectory[figDir]];
figSave[name_String, g_] := (Export[FileNameJoin[{figDir, name <> ".pdf"}], g]; g);
(* figShow prints every panel as its own output (at most 460 points wide, so that the no
   prints to PDF without scroll panes cutting the figures) and exports the composite fig
figShow[name_String, panels_List, composite_] := (
   Scan[Print, panels];
   Export[FileNameJoin[{figDir, name <> ".pdf"}], composite];
   Print[Style["saved: " <> FileNameJoin[{figDir, name <> ".pdf"}], Gray, 9]];);
(* number formatting without padding messages: n significant digits, integers when large
```

```
figNum[x_?NumericQ, n_Integer: 4] := StringReplace[
   If[Abs[x] >= 10^n, ToString[Round[x]], ToString[NumberForm[N[x], n]]], RegularExpression["\
figBlue = RGBColor[0.12, 0.38, 0.70]; figRed = RGBColor[0.84, 0.24, 0.24];

(* Parameter sets (concentrations in nM; k1, k3, k5, k7 in 1/(nM s); others in 1/s).
   figSet3: Iteration 1, row 96 (Class 5), the set of main-text Fig 3.
   figSet6: Iteration 3, row 5990 (Class 6), fully kinase-phosphatase symmetric. *)
figSet3 = <|"KTOT" -> 132.77894269306515, "PTOT" -> 16.907466299146936, "k1" -> 0.004470
figSet6 = <|"KTOT" -> 25.68678690797921, "PTOT" -> 25.68678690797921, "k1" -> 0.05546638

(* Constants of the inverse parametrisation, Eqs (7)–(9). *)
figConst[pr_Association] := Module[{m1, m3, m5, m7, a1, a2, r, al, be},
  m1 = (pr["k11"] + pr["k2"])/pr["k1"]; m3 = (pr["k33"] + pr["k4"])/pr["k3"];
  m5 = (pr["k55"] + pr["k6"])/pr["k5"]; m7 = (pr["k77"] + pr["k8"])/pr["k7"];
  a1 = pr["k2"] m7/(pr["k8"] m1); a2 = pr["k4"] m5/(pr["k6"] m3);
  r = pr["KTOT"]/pr["PTOT"];
  al = (a1/m3) (r pr["k4"]/pr["k6"] - 1); be = (1/m1) (r pr["k2"]/pr["k8"] - 1);
  <|"M1" -> m1, "M3" -> m3, "M5" -> m5, "M7" -> m7, "a1" -> a1, "a2" -> a2, "r" -> r,
    "alpha" -> al, "beta" -> be, "ustar" -> If[al != 0, -be/al, Infinity],
    "degenerate" -> (Abs[al r + be] <= 10^-9 (Abs[al] r + Abs[be]))|>];

(* Free substrate A(u), Eq. (8), and the full steady state at (u, A), Eq. (9). *)
figA[c_Association, u_] := (u - c["r"])/(u (c["alpha"] u + c["beta"]));
figState[pr_Association, c_Association, u_, a_] :=
 Module[{kk, pp, x1, x2, x3, x4, x5, x6},
  kk = pr["KTOT"]/(1 + a/c["M1"] + c["a1"] u a/c["M3"]); pp = kk/u;
  x1 = c["a1"] u a; x2 = c["a1"] c["a2"] u^2 a;
  x3 = a kk/c["M1"]; x4 = x1 kk/c["M3"]; x5 = x2 pp/c["M5"]; x6 = x1 pp/c["M7"];
  <|"u" -> u, "A" -> a, "K" -> kk, "P" -> pp, "x" -> {x1, x2, x3, x4, x5, x6},
    "ATOT" -> a + x1 + x2 + x3 + x4 + x5 + x6|>];

(* Reduced six-state model, Eqs (1)–(2); state order {AP, APP, AK, APK, APPC, APC}. *)
figRHS[pr_Association, at_, xs_List] := Module[{v, fA, fK, fP},
  fA = at - Total[xs]; fK = pr["KTOT"] - xs[[3]] - xs[[4]]; fP = pr["PTOT"] - xs[[5]] - xs[[6]];
  v = {pr["k1"] fA fK - pr["k11"] xs[[3]], pr["k2"] xs[[3]],
       pr["k3"] xs[[1]] fK - pr["k33"] xs[[4]], pr["k4"] xs[[4]],
       pr["k5"] xs[[2]] fP - pr["k55"] xs[[5]], pr["k6"] xs[[5]],
       pr["k7"] xs[[1]] fP - pr["k77"] xs[[6]], pr["k8"] xs[[6]]};
  {v[[2]] - v[[3]] + v[[6]] - v[[7]], v[[4]] - v[[5]], v[[1]] - v[[2]],
   v[[3]] - v[[4]], v[[5]] - v[[6]], v[[7]] - v[[8]]}];

(* Analytic Jacobian in the compact form of the notebook (support/jac6.m). *)
figJac[pr_Association, at_, xs_List] := Module[{x1, x2, x3, x4, x5, x6, s, cK, cP},
  {x1, x2, x3, x4, x5, x6} = xs; s = Total[xs] - at;
  cK = x3 + x4 - pr["KTOT"]; cP = x5 + x6 - pr["PTOT"];
  With[{k1 = pr["k1"], k11 = pr["k11"], k2 = pr["k2"], k3 = pr["k3"], k33 = pr["k33"],
        k4 = pr["k4"], k5 = pr["k5"], k55 = pr["k55"], k6 = pr["k6"], k7 = pr["k7"],
        k77 = pr["k77"], k8 = pr["k8"]},
   {{k3 cK + k7 cP, 0, k2 + x1 k3, k33 + x1 k3, k6 + x1 k7, k77 + x1 k7},
    {0, k5 cP, 0, k4, k55 + x2 k5, x2 k5},
    {k1 cK, k1 cK, k1 s - k11 - k2 + k1 cK, k1 s + k1 cK, k1 cK, k1 cK},
    {-k3 cK, 0, -x1 k3, -k4 - k33 - x1 k3, 0, 0},
    {0, -k5 cP, 0, 0, -k6 - k55 - x2 k5, -x2 k5},
```

```
    {-k7 cP, 0, 0, 0, -x1 k7, -k8 - k77 - x1 k7}}];
figStableQ[pr_Association, at_, xs_List] := Max[Re[Eigenvalues[N[figJac[pr, at, xs]]]]] < 0;

(* All positive steady states as branches of the parametrisation. *)
figPosQ[s_Association] := Min[s["x"]] > 0 && s["K"] > 0 && s["P"] > 0;
figUGrid[c_Association] := Module[{crit = Select[{c["r"], c["ustar"]}, 0 < # < Infinity &]},
  Union[10.^Range[-14., 10., 0.004],
   Flatten[Table[cc {1 - 10.^-k, 1 + 10.^-k}, {cc, crit}, {k, 1, 12}]]]];
figBranches[pr_Association] := Module[{c = figConst[pr], r, crit, us, pts, br},
  r = c["r"];
  If[TrueQ[c["degenerate"]],
   (* symmetric case: the line u = r (any A) and the curve A = 1/(alpha u) *)
   br = {figState[pr, c, r, #] & /@ (10.^Range[-8., 8., 0.004])};
   If[c["alpha"] > 0,
    AppendTo[br, figState[pr, c, #, 1/(c["alpha"] #)] & /@ (10.^Range[-6., 6., 0.002])]],
   (* general case: the intervals of {u > 0 : A(u) > 0} between r and u* *)
   crit = Select[{r, c["ustar"]}, 0 < # < Infinity &];
   us = Select[figUGrid[c], figA[c, #] > 0 &];
   pts = figState[pr, c, #, figA[c, #]] & /@ us;
   br = GatherBy[pts, Function[s, Count[crit, cc_ /; cc < s["u"]]]]];
  br = Function[b, Select[b, figPosQ]] /@ br;
  Select[br, Length[#] > 1 &]];

(* Folds: extrema of ATOT along a branch; branch point of the symmetric case. *)
figFoldsOf[branch_List] := Module[{g = #["ATOT"] & /@ branch, d},
  d = Differences[Sign[Differences[g]]];
  branch[[1 + Flatten[Position[d, z_ /; z != 0, {1}]]]]];
figBP[pr_Association] := With[{c = figConst[pr]},
  If[TrueQ[c["degenerate"]] && c["alpha"] > 0,
   figState[pr, c, c["r"], 1/c["alpha"]], Missing["NoBranchPoint"]]];
figSingular[pr_Association, br_List] :=
  Join[Flatten[figFoldsOf /@ br, 1], If[MissingQ[figBP[pr]], {}, {figBP[pr]}]];

(* Exact bifurcation diagram APP against ATOT: blue stable, red unstable, black dots
   folds and branch points. Options (PlotRange, PlotLabel, ...) are passed on. *)
figDiagram[pr_Association, opts___] := Module[{br = figBranches[pr], lab, st, un, sing, da
  lab = Flatten[Function[b, {#["ATOT"], #["x"][[2]], figStableQ[pr, #["ATOT"], #["x"]]} & /@
  st = Cases[lab, {a_, b_, True} :> {a, b}]; un = Cases[lab, {a_, b_, False} :> {a, b}];
  sing = {#["ATOT"], #["x"][[2]]} & /@ figSingular[pr, br];
  data = Select[{{st, Directive[figBlue, PointSize[0.006]]}, {un, Directive[figRed, PointSi
      {sing, Directive[Black, PointSize[0.02]]}}, Length[First[#]] > 0 &];
  ListLogLogPlot[data[[All, 1]], opts, PlotStyle -> data[[All, 2]],
   PlotRange -> {{10^-1, 10^6}, All}, Frame -> True, FrameLabel -> {"A_TOT (nM)", "APP (nM)
   ImageSize -> 440]];

(* Steady states at a given ATOT (roots of ATOT(u) = at on every branch). *)
figStatesAt[pr_Association, at_] := Module[{c = figConst[pr], br, out = {}, g, lu, s},
  br = figBranches[pr];
  Do[g = #["ATOT"] - at & /@ b;
   Do[If[g[[j]] g[[j + 1]] <= 0,
     AppendTo[out,
      If[TrueQ[c["degenerate"]], b[[j]],
       s = FindRoot[figState[pr, c, 10^lu, figA[c, 10^lu]]["ATOT"] == at,
```

```
        {lu, Log10[b[[j]]["u"]], Log10[b[[j + 1]]["u"]]}];
      figState[pr, c, 10^(lu /. s), figA[c, 10^(lu /. s)]]]],
   {j, Length[g] - 1}], {b, br}];
  SortBy[out, #["u"] &]];

(* Fold of the detached branch (minimum of ATOT(u) on the branch that does not reach u =
figDetachedFold[pr_Association] := Module[{c = figConst[pr], br, det, i0, lu, sol},
  If[TrueQ[c["degenerate"]], Return[Missing["NoDetachedBranch"]]];
  br = figBranches[pr];
  det = Select[br, Min[Abs[Log10[#["u"]/c["r"]]] & /@ #] > 10^-3 &];
  If[det === {}, Return[Missing["NoDetachedBranch"]]];
  det = First[det];
  i0 = First[Ordering[#["ATOT"] & /@ det, 1]];
  (* derivative-free search started from two points (no line search, no lstol message) *)
  sol = Quiet[FindMinimum[figState[pr, c, 10^lu, figA[c, 10^lu]]["ATOT"],
     {lu, Log10[det[[i0]]["u"]] - 0.01, Log10[det[[i0]]["u"]] + 0.01}], {FindMinimum::lstol}];
  figState[pr, c, 10^(lu /. sol[[2]]), figA[c, 10^(lu /. sol[[2]])]]];

(* Second derivatives and dF/dATOT in augmented coordinates (x1..x6, ATOT), and the
   right/left null vectors q, p of a singular Jacobian (|q| = 1, p.q = 1). *)
figHess[pr_Association, at_, xs_List] := Module[{z = Array[figZ, 7], f},
  f = figRHS[pr, z[[7]], z[[1 ;; 6]]];
  D[f, {z, 2}] /. Thread[z -> Append[xs, at]]];
figFA[pr_Association, at_, xs_List] := Module[{z7}, D[figRHS[pr, z7, xs], z7] /. z7 -> at];
figNull[jm_] := Module[{uu, ww, vv, q, p},
  {uu, ww, vv} = SingularValueDecomposition[N[jm]];
  q = vv[[All, -1]]; p = uu[[All, -1]]; q = q/Norm[q]; p = p/(p . q);
  {q, p, ww[[-1, -1]]/ww[[1, 1]]}];

(* Branch point: exact branches near the branch point in linear axes, with the two
   tangent directions given by the branching equation a xi^2 + 2 b xi eta + c eta^2 = 0
figBPPlot[pr_Association, title_] := Module[{bp, xs, at, app, jm, q, p, fa, mm, sol, v0,
   a, b, cc, dirs, lines, br, lab, st, un, tmax},
  bp = figBP[pr]; If[MissingQ[bp], Return[Missing["NoBranchPoint"]]];
  xs = bp["x"]; at = bp["ATOT"]; app = xs[[2]];
  jm = figJac[pr, at, xs]; {q, p} = Most[figNull[jm]]; fa = figFA[pr, at, xs];
  mm = ArrayFlatten[{{jm, Transpose[{p}]}, {{q}, 0}}];
  sol = LinearSolve[mm, Append[-fa, 0]]; v0 = Most[sol];
  h = figHess[pr, at, xs]; qb = Append[q, 0]; vb = Append[v0, 1];
  a = p . (qb . # . qb & /@ h); b = p . (qb . # . vb & /@ h); cc = p . (vb . # . vb & /@
  dirs = If[Abs[a] < 10^-4 Max[Abs[{b, cc}]],
    {{1, 0}, {-cc/(2 b), 1}},
    {{(-b + Sqrt[b^2 - a cc])/a, 1}, {(-b - Sqrt[b^2 - a cc])/a, 1}}];
  lines = Function[d, With[{w = d[[1]] qb + d[[2]] vb}, {w[[7]], w[[2]]}]] /@ dirs;
  br = figBranches[pr];
  lab = Flatten[Function[bb, {#["ATOT"], #["x"][[2]], figStableQ[pr, #["ATOT"], #["x"]]} & /@
  lab = Select[lab, 0.4 at < #[[1]] < 1.6 at && #[[2]] < 3 app &];
  st = Cases[lab, {x_, y_, True} :> {x, y}]; un = Cases[lab, {x_, y_, False} :> {x, y}];
  Show[ListPlot[Select[{st, un}, Length[#] > 0 &],
    PlotStyle -> {Directive[figBlue, PointSize[0.006]], Directive[figRed, PointSize[0.006]]
   Graphics[{Black, Dashed, Thickness[0.004],
     Function[l, tmax = Min[If[Abs[l[[1]]] > 0, 0.5 at/Abs[l[[1]]], Infinity],
         If[Abs[l[[2]]] > 0, 1.2 app/Abs[l[[2]]], Infinity]];
```

```
      Line[{{at, app} - tmax l, {at, app} + tmax l}] /@ lines,
    PointSize[0.025], Point[{at, app}]}],
  PlotRange -> {{0.4 at, 1.6 at}, {0, 3 app}}, Frame -> True,
  FrameLabel -> {"A_TOT (nM)", "APP (nM)"}, ImageSize -> 440,
  PlotLabel -> Style[Column[{title, Row[{"a = ", ScientificForm[a, 3], ",  b = ", Scienti
       ",  c = ", ScientificForm[cc, 3]}], Row[{"discriminant b^2 - a c = ", ScientificFo
       "  (dashed: tangents)"}]}, Alignment -> Center], 10]];
```

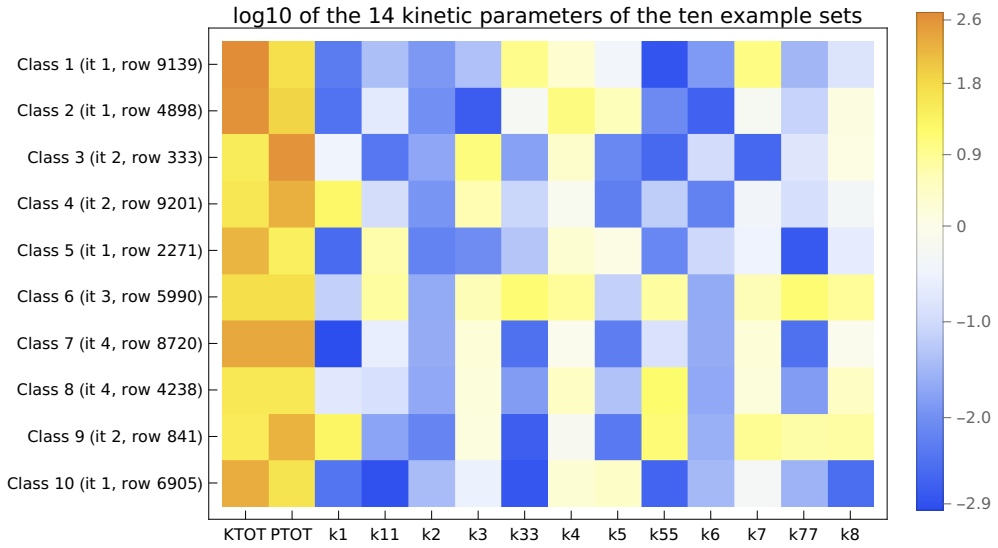
In[241]:=

```
(* The ten example sets of the notebook (same values as exampleRows of Section 2). *)
figExampleRows = {
  <|"Class" -> 1, "Label" -> "LP_S_ONLY","Iteration" -> 1, "ExcelRow" -> 9139, "KTOT" -
  <|"Class" -> 2, "Label" -> "LP_S_DETACHED","Iteration" -> 1, "ExcelRow" -> 4898, "KTO
  <|"Class" -> 3, "Label" -> "LP_Z_ONLY","Iteration" -> 2, "ExcelRow" -> 333, "KTOT" ->
  <|"Class" -> 4, "Label" -> "LP_Z_DETACHED","Iteration" -> 2, "ExcelRow" -> 9201, "KTO
  <|"Class" -> 5, "Label" -> "LP_DETACHED_ONLY","Iteration" -> 1, "ExcelRow" -> 2271, "
  <|"Class" -> 6, "Label" -> "BP_ONLY", "Iteration" -> 3, "ExcelRow" -> 5990, "KTOT" ->
  <|"Class" -> 7, "Label" -> "LP_BP_LPgreaterBP","Iteration" -> 4, "ExcelRow" -> 8720,
  <|"Class" -> 8, "Label" -> "LP_BP_LPlessBP","Iteration" -> 4, "ExcelRow" -> 4238, "KT
  <|"Class" -> 9, "Label" -> "NO_SING_biphasic","Iteration" -> 2, "ExcelRow" -> 841, "K
  <|"Class" -> 10, "Label" -> "NO_SING_monotone","Iteration" -> 1, "ExcelRow" -> 6905,
figParNames = {"KTOT", "PTOT", "k1", "k11", "k2", "k3", "k33", "k4", "k5", "k55", "k6",
figXY[d_Association] := {Log10[d["k8"] d["PTOT"]/(d["k2"] d["KTOT"])], Log10[d["k4"] d["KTOT"]/(d[
```

Added figure. Plot: log10 of the 14 kinetic parameters of the ten example sets (rows: class, sampling iteration and row of ML_Ready_Dataset_v3_iter<i>.csv). The symmetric sets of Iterations 3 and 4 show the repeated kinase/phosphatase values. Saved as E1_example_parameters.pdf.

In[244]:=

```
(* Plot for Section 1: parameter values of the ten example sets *)
With[{g = MatrixPlot[Table[Log10[figRow[n]],{figRow, figExampleRows}, {n, figParNames}],
  ColorFunction -> "TemperatureMap", PlotLegends -> Automatic,
  FrameTicks -> {{Transpose[{Range[10], ("Class " <> ToString[#["Class"]] <> " (it " <> ToSt
          ", row " <> ToString[#["ExcelRow"]] <> ")") & /@ figExampleRows}], None},
    {Transpose[{Range[14], figParNames}], None}},
  PlotLabel -> "log10 of the 14 kinetic parameters of the ten example sets", ImageSize
 figShow["E1_example_parameters",{g}, g]]
```



saved: /wolframcloud/userfiles/9ff/9ffe219c-88d9-4035-be0c-26993d670000/Files/Harika/Derivation_Notebooks/Final/plots/E1_example_parameters.pdf

2. Enter the Excel parameter values

The values below were read from the Excel workbook and are embedded so this notebook runs without external file paths. The iteration and Excel row identify each source example.

In[245]:=

```
exampleRows = {
 <|
 "Class" → 1, "Label" → "LP_S_ONLY", "Iteration" → 1, "ExcelRow" → 9139,
 "KTOT" → 493.805893340563, "PTOT" → 23.50939993607158,
 "k1" → 0.004556397728147515, "k11" → 0.02848578336741475,
 "k2" → 0.01016640884850226, "k3" → 0.02861071572214909,
 "k33" → 5.035605546359657, "k4" → 1.527359612033014,
 "k5" → 0.3611327431144362, "k55" → 0.001336526818424585,
 "k6" → 0.01040756134021452, "k7" → 6.056967399200773,
 "k77" → 0.02117321427044033, "k8" → 0.1113684206503778
 |>,
 <|
 "Class" → 2, "Label" → "LP_S_DETACHED", "Iteration" → 1, "ExcelRow" → 4898,
 "KTOT" → 439.9290776366189, "PTOT" → 39.0306419246769,
 "k1" → 0.003453899266497233, "k11" → 0.1499445146392653,
 "k2" → 0.006987683164556002, "k3" → 0.00173856060853252,
 "k33" → 0.4866440128720959, "k4" → 6.077541939958889,
 "k5" → 2.866036136127631, "k55" → 0.0060796380672921,
 "k6" → 0.002193909346714001, "k7" → 0.5173448748027955,
 "k77" → 0.06169229498938726, "k8" → 1.159443828038747
 |>,
 <|
```

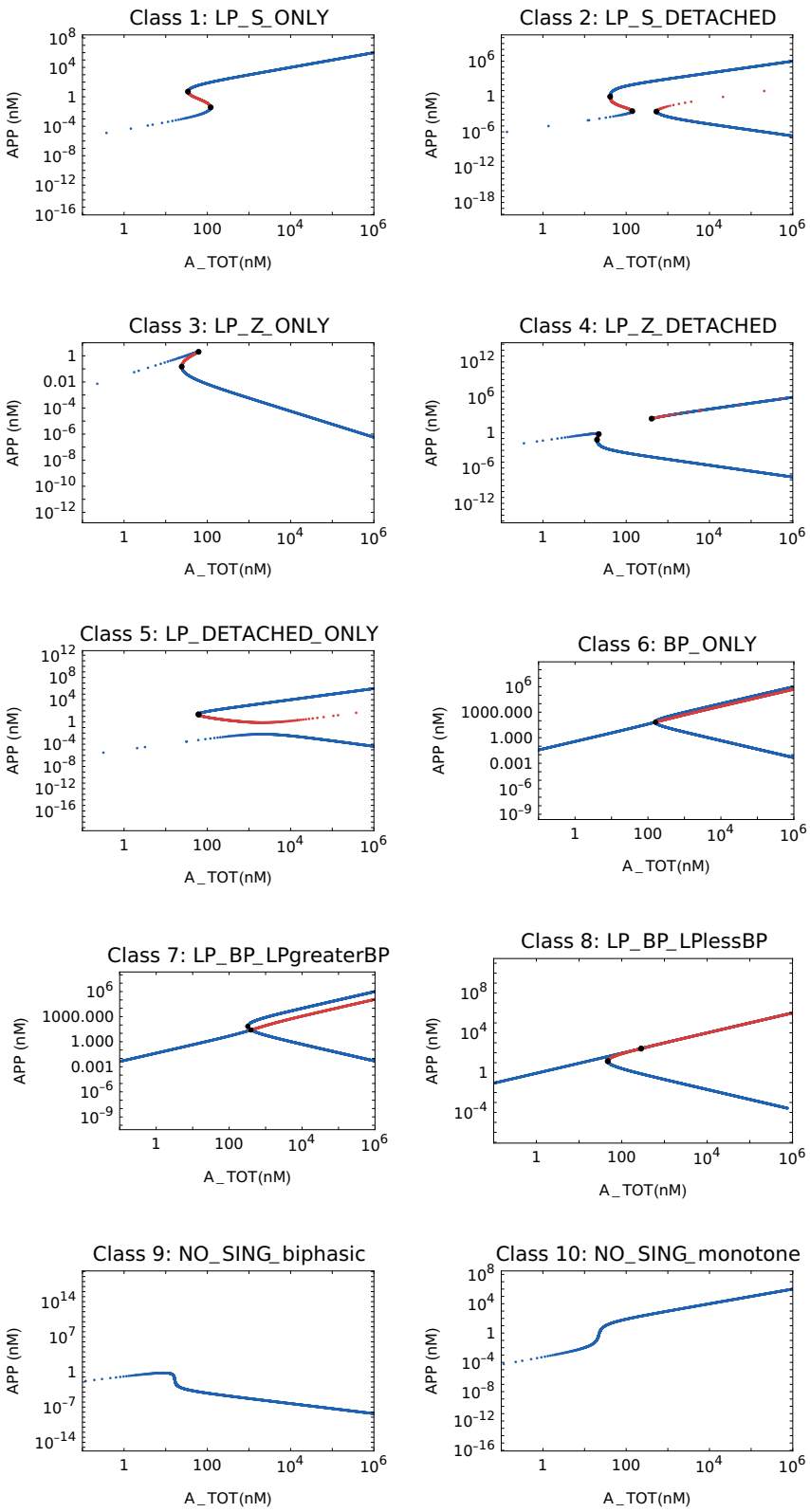
```
 "Class" → 3, "Label" → "LP_Z_ONLY", "Iteration" → 2, "ExcelRow" → 333,
 "KTOT" → 16.27104195701832, "PTOT" → 436.5581812313756,
 "k1" → 0.304733968660903, "k11" → 0.004002842022553523,
 "k2" → 0.0143715542510764, "k3" → 6.151648936337424,
 "k33" → 0.01289653599175536, "k4" → 1.605843942061278,
 "k5" → 0.005779405834586354, "k55" → 0.002643760844558748,
 "k6" → 0.09052437704755904, "k7" → 0.002580913371630385,
 "k77" → 0.1336195267277172, "k8" → 1.099942782887686
|>,
<|
 "Class" → 4, "Label" → "LP_Z_DETACHED", "Iteration" → 2, "ExcelRow" → 9201,
 "KTOT" → 18.60996218128323, "PTOT" → 168.9019424784452,
 "k1" → 9.070357210279901, "k11" → 0.09286962665176958,
 "k2" → 0.009243334098956595, "k3" → 3.11670904402922,
 "k33" → 0.06634850081998682, "k4" → 0.6348674363282099,
 "k5" → 0.004608629744578452, "k55" → 0.05075798198733661,
 "k6" → 0.005245313890869659, "k7" → 0.3492774124255668,
 "k77" → 0.09345314172914013, "k8" → 0.3904738324764311
|>,
<|
 "Class" → 5, "Label" → "LP_DETACHED_ONLY", "Iteration" → 1, "ExcelRow" → 2271,
 "KTOT" → 160.0450231416401, "PTOT" → 14.45309061236188,
 "k1" → 0.002746476259979306, "k11" → 3.512235311587292,
 "k2" → 0.005271742406959697, "k3" → 0.006214644645492652,
 "k33" → 0.03217641067548698, "k4" → 1.4019149764765,
 "k5" → 1.086009486434319, "k55" → 0.005546276360329681,
 "k6" → 0.0697149759516603, "k7" → 0.2860816059650637,
 "k77" → 0.001624209770233322, "k8" → 0.1680149389739284
|>,
<|
 "Class" → 6, "Label" → "BP_ONLY", "Iteration" → 3, "ExcelRow" → 5990,
 "KTOT" → 25.68678690797921, "PTOT" → 25.68678690797921,
 "k1" → 0.0554663818609845 4, "k11" → 3.709922237775117,
 "k2" → 0.01505315498170104, "k3" → 2.969551126146651,
 "k33" → 6.500575890125845, "k4" → 3.951959656644954,
 "k5" → 0.05546638186098454, "k55" → 3.709922237775117,
 "k6" → 0.01505315498170104, "k7" → 2.969551126146651,
 "k77" → 6.500575890125845, "k8" → 3.951959656644954
|>,
<|
 "Class" → 7, "Label" → "LP_BP_LPgreaterBP", "Iteration" → 4, "ExcelRow" → 8720,
 "KTOT" → 181.5325803888077, "PTOT" → 181.5325803888077,
 "k1" → 0.00108573147823123, "k11" → 0.1872188229592035,
 "k2" → 0.01518329994945931, "k3" → 1.388993144845929,
 "k33" → 0.003370893693737047, "k4" → 0.7956746290932591,
```

```
 "k5" → 0.004584062981815904, "k55" → 0.09833452155338396,
 "k6" → 0.01518329994945931, "k7" → 1.388993144845929,
 "k77" → 0.003370893693737047, "k8" → 0.7956746290932591
 |>,
 <|
 "Class" → 8, "Label" → "LP_BP_LPlessBP", "Iteration" → 4, "ExcelRow" → 4238,
 "KTOT" → 18.48762101143704, "PTOT" → 18.48762101143704,
 "k1" → 0.1443995887474054, "k11" → 0.09477266979435336,
 "k2" → 0.01448604494013809, "k3" → 1.298327879297094,
 "k33" → 0.01077780806047449, "k4" → 1.946054105157064,
 "k5" → 0.02882246689490239, "k55" → 7.571181637920173,
 "k6" → 0.01448604494013809, "k7" → 1.298327879297094,
 "k77" → 0.01077780806047449, "k8" → 1.946054105157064
 |>,
 <|
 "Class" → 9, "Label" → "NO_SING_biphasic", "Iteration" → 2, "ExcelRow" → 841,
 "KTOT" → 16.45004390712542, "PTOT" → 161.9860935476338,
 "k1" → 9.766128563965379, "k11" → 0.014017206364713,
 "k2" → 0.005303749556216574, "k3" → 1.16793115477964,
 "k33" → 0.001813276829464453, "k4" → 0.5561369004360806,
 "k5" → 0.004012213324140902, "k55" → 6.388962505346301,
 "k6" → 0.01781571617452874, "k7" → 4.396316952773555,
 "k77" → 3.556285188756318, "k8" → 3.598274835776406
 |>,
 <|
 "Class" → 10, "Label" → "NO_SING_monotone", "Iteration" → 1, "ExcelRow" → 6905,
 "KTOT" → 170.4408392232811, "PTOT" → 21.56696956624577,
 "k1" → 0.00360831667340678, "k11" → 0.001256210392239927,
 "k2" → 0.02607399683590078, "k3" → 0.2289408368454112,
 "k33" → 0.001395401559920961, "k4" → 1.399293397884767,
 "k5" → 1.809372168600029, "k55" → 0.002232525035919418,
 "k6" → 0.02302089515896241, "k7" → 0.4168139745011278,
 "k77" → 0.02000049274262368, "k8" → 0.002897002934009536
 |>
};
```

Added figure. Plot: the exact bifurcation diagram APP against A_TOT of every example set, computed from the inverse parametrisation (blue stable, red unstable, black folds and branch points; window 0.1 to 10^6 nM). Each diagram shows the features of its class. Saved as E2_example_bifurcation_diagrams.pdf.

In[246]:=

```
(* Plot for Section 2: exact bifurcation diagram of every example set *)
figExampleDiagrams = figDiagram[#, ImageSize -> 300,
     PlotLabel -> Style["Class " <> ToString[#["Class"]] <> ": " <> #["Label"], 11]] & /@
figShow["E2_example_bifurcation_diagrams",GraphicsRow[#, ImageSize -> 460] & /@ Partiti
 GraphicsGrid[Partition[figExampleDiagrams, 5], ImageSize -> 1600]]
```



saved: /wolframcloud/userfiles/9ff/9ffe219c-88d9-4035-be0c-26993d670000/Files/Harika/Derivation_Notebooks/Final/plots/E2_example_bifurcation_diagrams.pdf

3. Calculate Michaelis constants and detached-branch inequalities

For each example, compute M1, M3, M5, and M7 from the kinetic rates. Also evaluate both strict inequalities in the detached-branch condition. Passing this condition indicates the detached-branch geometry; it does not by itself determine the complete bifurcation label.

In[248]:=

```
ToExpression["derivedExamples =
  Map[Function[d,Module[{m1,m3,m5,m7,det1,det2},m1=(d[\"k11\"]+d[\"k2\"])/d[\"k1\"];m3=(d[
  \"k33\"]+d[\"k4\"])/d[\"k3\"];m5=(d[\"k55\"]+d[\"k6\"])/d[\"k5\"];m7=(d[\"k77\"]+d[\"k8\"])/
  d[\"k7\"];det1=d[\"k8\"]
  d[\"PTOT\"]>d[\"k2\"] d[\"KTOT\"];det2=d[\"k4\"] d[\"KTOT\"]>d[\"k6\"]
  d[\"PTOT\"];\nJoin[d,<|\"M1\"->m1,\"M3\"->m3,\"M5\"->m5,\"M7\"->m7,\"
  DetachedInequality1\"->det1,\"DetachedInequality2\"->det2,\"
  BothDetachedConditions\"->(det1&&det2)|>]]],exampleRows];\nDataset[Map[KeyTake[#,
  {\"Class\",\"Label\",\"Iteration\",\"ExcelRow\",\"M1\",\"M3\",\"M5\",\"M7\",\"
  DetachedInequality1\",\"DetachedInequality2\",\"BothDetachedConditions\"}]&,
  derivedExamples]]"]
```

Out[248]=

Class	Label	Iteration	ExcelRow	M1	M3	M5
1	LP_S_ONLY	1	9139	8.48306	229.388	0.0
2	LP_S_DETACHED	1	4898	45.4362	3775.64	0.0
3	LP_Z_ONLY	2	333	0.0602965	0.263139	16
4	LP_Z_DETACHED	2	9201	0.0112579	0.224986	12
5	LP_DETACHED_ONLY	1	2271	1280.73	230.76	0.0
6	BP_ONLY	3	5990	67.1574	3.5199	67
7	LP_BP_LPgreaterBP	4	8720	186.42	0.57527	24
8	LP_BP_LPlessBP	4	4238	0.756641	1.50719	26
9	NO_SING_biphasic	2	841	0.00197836	0.477725	15
10	NO_SING_monotone	1	6905	7.57423	6.11813	0.0

columns 1–10 of **11**

In[249]:=

```
derivedExamples = Map[
 Function[d,
  Module[
   {m1, m3, m5, m7, det1, det2},
   m1 = (d["k11"] + d["k2"])/d["k1"];
   m3 = (d["k33"] + d["k4"])/d["k3"];
   m5 = (d["k55"] + d["k6"])/d["k5"];
   m7 = (d["k77"] + d["k8"])/d["k7"];
       det1 = d["k8"] × d["PTOT"] > d["k2"] × d["KTOT"];
   det2 = d["k4"] × d["KTOT"] > d["k6"] × d["PTOT"];
        Join[
    d,
    <|
     "M1" → m1,
     "M3" → m3,
     "M5" → m5,
     "M7" → m7,
     "DetachedInequality1" → det1,
     "DetachedInequality2" → det2,
     "BothDetachedConditions" → (det1 && det2)
    |>
   ]
  ]
 ],
 exampleRows
];
```

In[250]:=

In[251]:=

In[252]:=

In[253]:=

```
Dataset[
 Map[
  KeyTake[
   #,
   {
    "Class", "Label", "Iteration", "ExcelRow",
    "M1", "M3", "M5", "M7",
    "DetachedInequality1", "DetachedInequality2",
    "BothDetachedConditions"
   }
  ] &,
  derivedExamples
 ]
]
```

Out[253]=

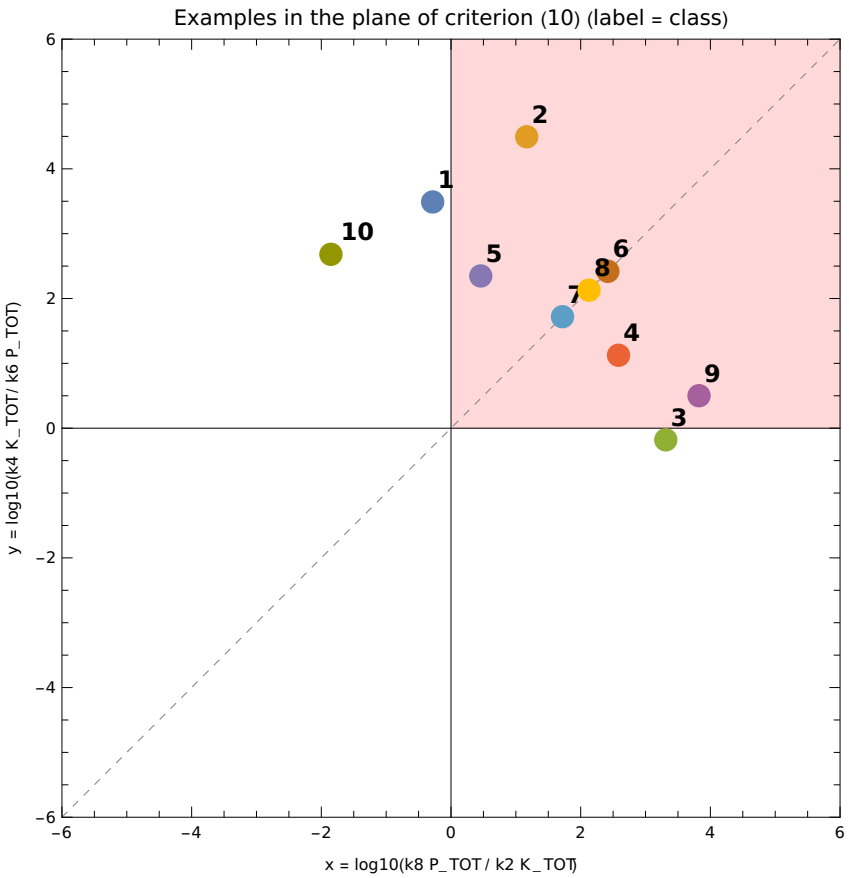
Class	Label	Iteration	ExcelRow	M1	M3	M5
1	LP_S_ONLY	1	9139	8.48306	229.388	0.0
2	LP_S_DETACHED	1	4898	45.4362	3775.64	0.0
3	LP_Z_ONLY	2	333	0.0602965	0.263139	16
4	LP_Z_DETACHED	2	9201	0.0112579	0.224986	12
5	LP_DETACHED_ONLY	1	2271	1280.73	230.76	0.0
6	BP_ONLY	3	5990	67.1574	3.5199	67
7	LP_BP_LPgreaterBP	4	8720	186.42	0.57527	24
8	LP_BP_LPlessBP	4	4238	0.756641	1.50719	26
9	NO_SING_biphasic	2	841	0.00197836	0.477725	15
10	NO_SING_monotone	1	6905	7.57423	6.11813	0.0

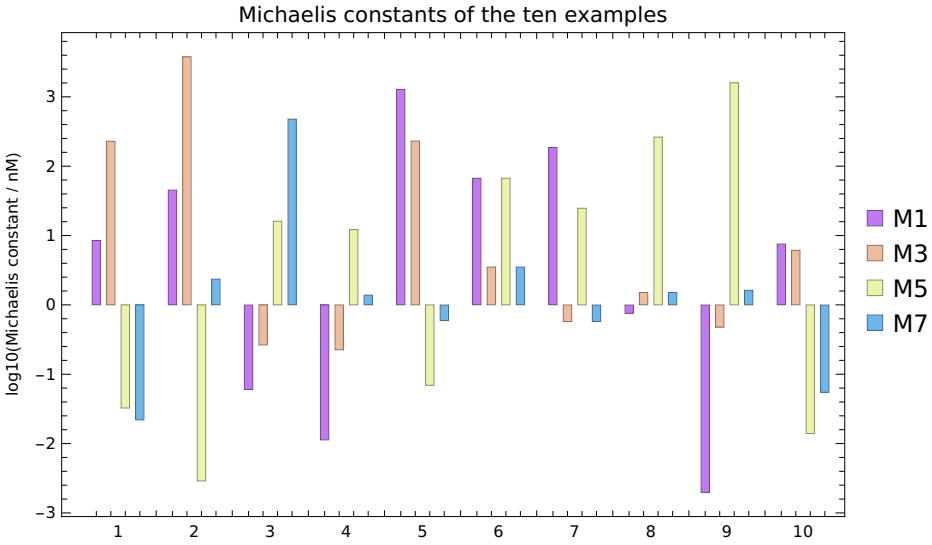
columns 1–10 of **11**

Added figure. Plots: (left) the example sets in the plane of criterion (10), x = log10(k8 P_TOT/(k2 K_TOT)) and y = log10(k4 K_TOT/(k6 P_TOT)); the shaded quadrant x > 0, y > 0 is where a detached branch exists. Classes 2, 4 and 5 lie inside it; Class 9 also lies inside but its detached fold is above the window (A_TOT > 10^6), so it is labelled by its main branch; the symmetric Classes 6 to 8 lie on the diagonal x = y. (Right) log10 of the four Michaelis constants of each example. Saved as E3_criterion_-plane_and_Michaelis_constants.pdf.

In[254]:=

```
(* Plot for Section 3: criterion (10) plane and Michaelis constants *)
Module[{xy = figXY /@ figExampleRows, cls = Lookup[figExampleRows, "Class"], lim, plane,
 lim = Ceiling[Max[Abs[Flatten[xy]]]] + 1;
 plane = Graphics[{RGBColor[1, 0.85, 0.85], Rectangle[{0, 0}, {lim, lim}], Black,
    Line[{{-lim, 0}, {lim, 0}}], Line[{{0, -lim}, {0, lim}}], Dashed, Gray, Line[{{-lim, -lim},
    MapThread[{ColorData[97][#2], PointSize[0.03], Point[#1], Black, Text[Style[ToString[#2
   Frame -> True, PlotRange -> {{-lim, lim}, {-lim, lim}}, AspectRatio -> 1,
   FrameLabel -> {"x = log10(k8 P_TOT / k2 K_TOT)", "y = log10(k4 K_TOT / k6 P_TOT)"},
   PlotLabel -> "Examples in the plane of criterion (10) (label = class)", ImageSize -> 4
 ms = Log10[Values[KeyTake[figConst[#], {"M1", "M3", "M5", "M7"}]]] & /@ figExampleRows;
 bars = BarChart[ms, ChartLabels -> {(Rotate[ToString[#], 0] & /@ cls), None},
   ChartLegends -> {"M1", "M3", "M5", "M7"}, ChartStyle -> "Pastel", Frame -> True,
   FrameLabel -> {None, "log10(Michaelis constant / nM)"},
   PlotLabel -> "Michaelis constants of the ten examples", ImageSize -> 440];
 figShow["E3_criterion_plane_and_Michaelis_constants",{plane, bars}, Row[{plane, bars}, '
```





saved:
/wolframcloud/userfiles/9ff/9ffe219c-88d9-4035-be0c-26993d670000/Files/Harika/Derivation_Notebooks/Final/plots/E3_criterion_plane_and_Michaelis_constants.pdf

4. Check the symmetric pitchfork and crossing-branch slope examples

The analytical crossing-slope formula predicts a zero slope for the symmetric Class 6 example, negative slope when M5 < M1 (Class 7), and positive slope when M5 > M1 (Class 8). The table evaluates the formula using the Excel parameters.

In[255]:=

```
slopeExamples = Map[
 Function[d,
  Module[
   {m1, m3, m5, den, slope, expected, observedSign},
        m1 = (d["k11"] + d["k2"])/d["k1"];
   m3 = (d["k33"] + d["k4"])/d["k3"];
   m5 = (d["k55"] + d["k6"])/d["k5"];
      den =
    d["KTOT"]*(d["k4"]^2 - d["k2"]^2) +
    d["k4"]^2*(m1 + m5) +
    d["k2"]*d["k4"]*m3;
        slope = If[
    den == 0,
    Indeterminate,
    d["k4"]^2*(m5 - m1)/den
   ];
      expected = Which[
    d["Class"] == 6, "symmetric",
    d["Class"] == 7, "negative",
    d["Class"] == 8, "positive",
    True, "not applicable"
   ];
       observedSign = Which[
    !NumericQ[slope], "undefined",
    Abs[N[slope]] < 10^-10, "zero",
    N[slope] < 0, "negative",
    N[slope] > 0, "positive",
    True, "undefined"
   ];
      Join[
    d,
    <|
     "M1" → m1,
     "M3" → m3,
     "M5" → m5,
```

```
        "SlopeEq13" → slope,
        "ExpectedSlopeFromClass" → expected,
        "ObservedSlopeSign" → observedSign,
        "M5LessThanM1" → (m5 < m1),
        "M5GreaterThanM1" → (m5 > m1)
       |>
     ]
    ]
   ],
   Select[exampleRows, MemberQ[{6, 7, 8}, #["Class"]] &]
  ];
```

In[256]:=

In[257]:=

In[258]:=

In[259]:=

In[260]:=

In[261]:=

In[262]:=

```
Dataset[
 Map[
  KeyTake[
   #,
   {
    "Class", "Label", "Iteration", "ExcelRow",
    "M1", "M5", "SlopeEq13",
    "ExpectedSlopeFromClass", "ObservedSlopeSign",
    "M5LessThanM1", "M5GreaterThanM1"
   }
  ] &,
  slopeExamples
 ]
]
```

Out[262]=

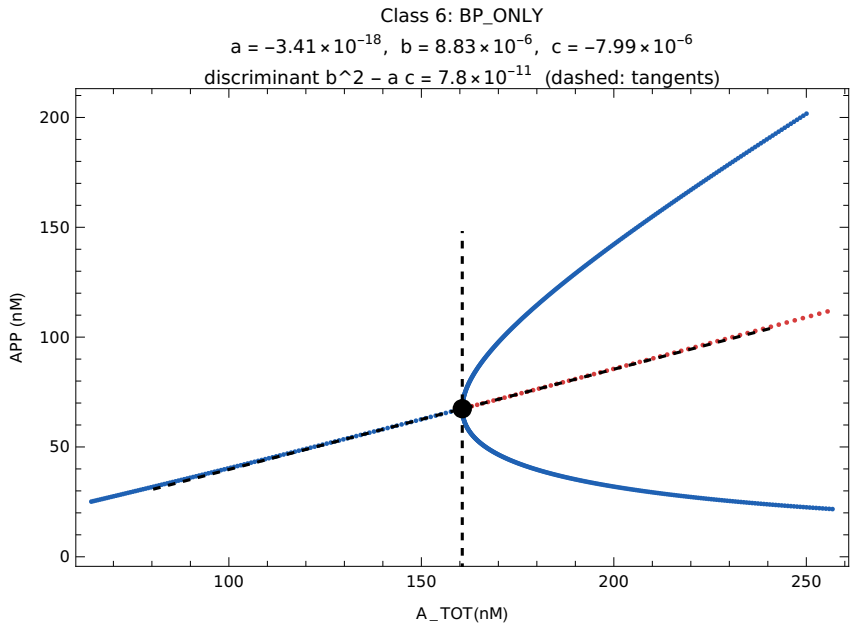
Class	Label	Iteration	ExcelRow	M1	M5	Sl
6	BP_ONLY	3	5990	67.1574	67.1574	0.(
7	LP_BP_LPgreaterBP	4	8720	186.42	24.7636	−0
8	LP_BP_LPlessBP	4	4238	0.756641	263.186	0.9

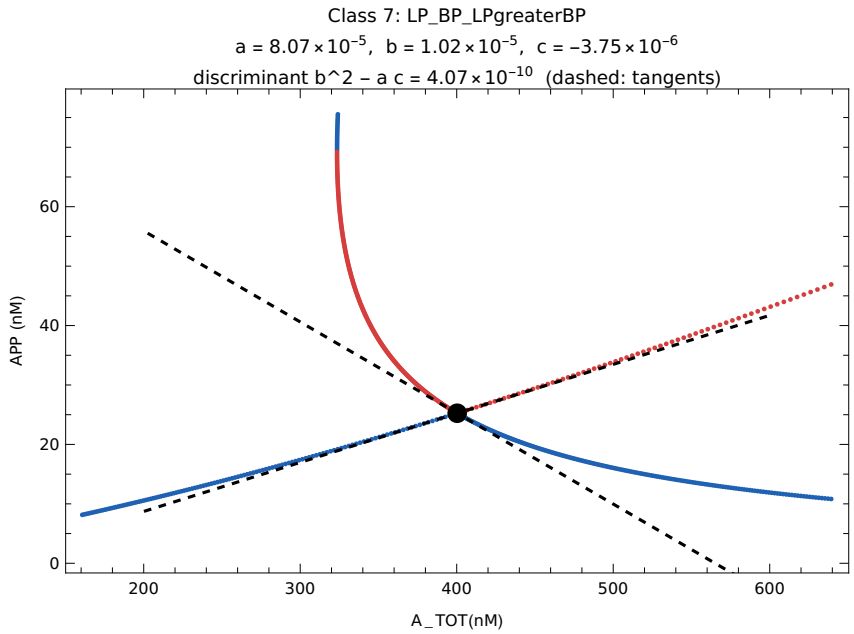
columns 1–10 of **11**

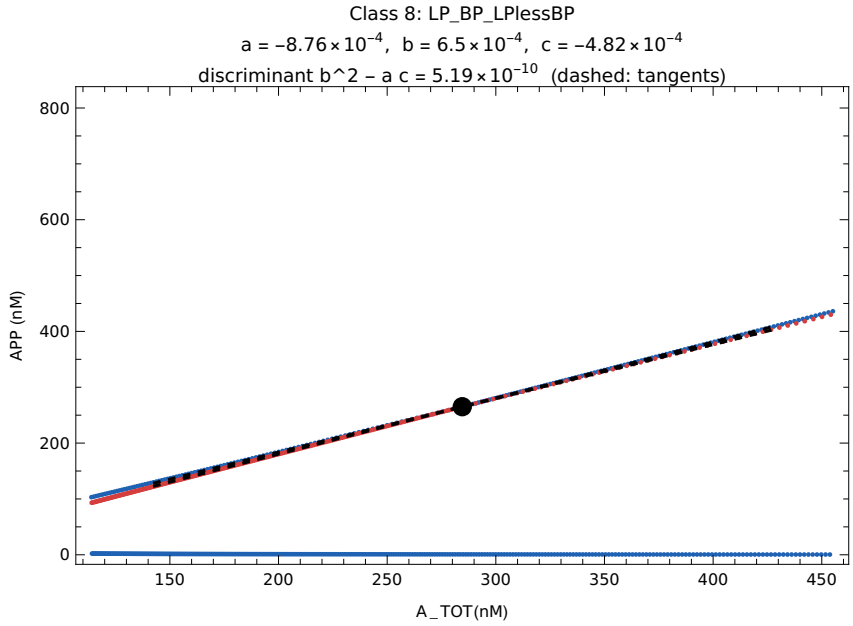
Added figure. Plots for the Class 6, 7 and 8 examples: (top) the exact branches near the branch point with the tangents of the branching equation (pitchfork for Class 6; transcritical crossings of opposite orientation for Classes 7 and 8); (bottom) the slope of Eq. (13) next to the slope d ln A_TOT / d ln u at u = 1 computed numerically from the parametrisation. Saved as E4_branch_points_and_slopes.pdf.

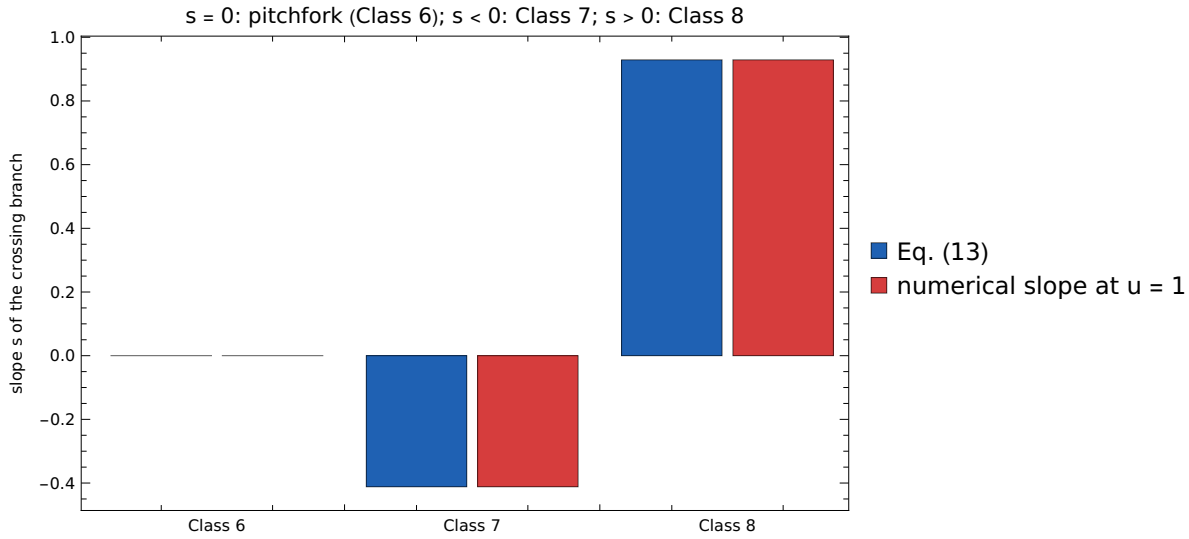
In[263]:=

```
(* Plot for Section 4: branch points of Classes 6, 7, 8 and the crossing-branch slope *)
Module[{ex = Select[figExampleRows, MemberQ[{6, 7, 8}, #["Class"]] &], zooms, s13, snum, ch
 zooms = figBPPlot[#, "Class " <> ToString[#["Class"]] <> ": " <> #["Label"]] & /@ ex;
 s13 = With[{c = figConst[#]}, #["k4"]^2 (c["M5"] - c["M1"])/(#["KTOT"] (#["k4"]^2 - #["k2"]^2)
 snum = With[{c = figConst[#], hh = 10^-5},
     (Log[figState[#, c, c["r"] (1 + hh), 1/(c["alpha"] c["r"] (1 + hh))]["ATOT"]] -
       Log[figState[#, c, c["r"] (1 - hh), 1/(c["alpha"] c["r"] (1 - hh))]["ATOT"]])/(2 hh)] & /@
 chart = BarChart[Transpose[{s13, snum}], ChartLabels -> {("Class " <> ToString[#["Class"]]
   ChartLegends -> {"Eq. (13)", "numerical slope at u = 1"}, ChartStyle -> {figBlue, figR
   FrameLabel -> {None, "slope s of the crossing branch"},
   PlotLabel -> "s = 0: pitchfork (Class 6); s < 0: Class 7; s > 0: Class 8", ImageSize
 figShow["E4_branch_points_and_slopes",Append[zooms, chart], Column[{GraphicsRow[zooms, I
```


Class 6: BP_ONLY
a = –3.41×10^–18, b = 8.83×10^–6, c = –7.99×10^–6
discriminant b^2 – a c = 7.8×10^–11 (dashed: tangents)
APP (nM)
A_TOT(nM)
0
50
100
150
200
100
150
200
250


Class 7: LP_BP_LPgreaterBP
a = 8.07×10^–5, b = 1.02×10^–5, c = –3.75×10^–6
discriminant b^2 – a c = 4.07×10^–10 (dashed: tangents)
APP (nM)
A_TOT(nM)
0
20
40
60
200
300
400
500
600





saved:
/wolframcloud/userfiles/9ff/9ffe219c-88d9-4035-be0c-26993d670000/Files/Harika/Derivation_Notebooks/Final/plots/E4_branch_points_and_slopes.pdf

5. Interpret what the numerical checks establish

These are checks of the analytical inequalities and slope sign on labeled examples from the Excel data. The detached-branch inequalities are geometric conditions rather than complete class classifiers. Class labels remain those assigned by the exact bifurcation pipeline.

In[264]:=

```
summary = <|
 "ExampleCount" → Length[exampleRows],
 "ClassesIncluded" → Sort[Lookup[exampleRows, "Class"]],
  "Class6SymmetricM1M5" → With[
 {
  d = First @ Select[
   derivedExamples,
   #["Class"] == 6 &
  ]
 },
 Chop[N[d["M1"] - d["M5"]]] == 0
],
  "Class7HasM5LessThanM1" → With[
 {
  d = First @ Select[
   derivedExamples,
   #["Class"] == 7 &
  ]
 },
 d["M5"] < d["M1"]
],
  "Class8HasM5GreaterThanM1" → With[
 {
  d = First @ Select[
   derivedExamples,
   #["Class"] == 8 &
  ]
 },
 d["M5"] > d["M1"]
]
|>;
```

In[265]:=

In[266]:=

In[267]:=

In[268]:=

```
summary
```

Out[268]=

```
<|ExampleCount → 10, ClassesIncluded → {1, 2, 3, 4, 5, 6, 7, 8, 9, 10},
 Class6SymmetricM1M5 → True, Class7HasM5LessThanM1 → True,
 Class8HasM5GreaterThanM1 → True|>
```

Added figure. Plot: for every example, what the inverse parametrisation finds in the window A_TOT <= 10^6 nM (number of folds on the main branch, whether a detached branch with a fold in the window exists, whether there is a branch point) next to criterion (10). The pattern of each row matches the definition of its class. Saved as E5_summary_of_checks.pdf.

In[269]:=

```
(* Plot for Section 5: summary of what the parametrisation finds for each example *)
Module[{rows, tick, grid},
 tick[b_] := If[TrueQ[b], Style["yes", Darker[Green], Bold], Style["no", Gray]];
 rows = Function[d, Module[{c = figConst[d], br, main, det, foldsMain, detFold, bp, crit}
     br = figBranches[d];
     main = Select[br, Min[Abs[Log10[#["u"]/c["r"]]] & /@ #] <= 10^-3 &];
     det = Select[br, Min[Abs[Log10[#["u"]/c["r"]]] & /@ #] > 10^-3 &];
     foldsMain = If[TrueQ[c["degenerate"]], {}, Select[Flatten[figFoldsOf /@ main, 1], #["A
     detFold = If[TrueQ[c["degenerate"]], {}, Select[Flatten[figFoldsOf /@ det, 1], #["ATOT
     bp = figBP[d];
     crit = And @@ Thread[figXY[d] > 0];
     {d["Class"], d["Label"], Length[foldsMain], tick[Length[detFold] > 0],
      tick[! MissingQ[bp] && bp["ATOT"] <= 10^6],
      If[TrueQ[c["degenerate"]], Length[Select[Flatten[figFoldsOf /@ Rest[br], 1], Abs[#["u"]
      tick[crit]}] /@ figExampleRows;
 grid = Style[Grid[Prepend[rows, Style[#, Bold] & /@ {"Class", "Label", "folds on\nmain b
      "branch point\nin window", "folds on\ncrossing branch", "criterion\n(10)"}],
   Frame -> All, Background -> {None, {LightGray}}, Spacings -> {0.8, 0.6}, Alignment -> (
 grid = Labeled[grid, Style["What the inverse parametrisation finds (window A_TOT <= 10^6
 figShow["E5_summary_of_checks",{grid}, grid]]
```

What the inverse parametrisation finds (window A_TOT <= 10^6 nM)

Class	Label	folds on main branch	detached fold in window	branch point in window	folds on crossing branch	criterion (10)
1	LP_S_ONLY	2	no	no	-	no
2	LP_S_DETACHED	2	yes	no	-	yes
3	LP_Z_ONLY	2	no	no	-	no
4	LP_Z_DETACHED	2	yes	no	-	yes
5	LP_DETACHED_ONLY	0	yes	no	-	yes
6	BP_ONLY	0	no	yes	0	yes
7	LP_BP_LPgreaterBP	0	no	yes	1	yes
8	LP_BP_LPlessBP	0	no	yes	1	yes
9	NO_SING_biphasic	0	no	no	-	yes
10	NO_SING_monotone	0	no	no	-	no

saved:
/wolframcloud/userfiles/9ff/9ffe219c-88d9-4035-be0c-26993d670000/Files/Harika/Derivation_Notebooks/Final/plots/E5_summary_of_checks.pdf